Jetson Orin Nano + Femto Mega 复现 ORB-SLAM2 RGB-D 房间三维重建教程 🧭



本文记录如何在 **Jetson Orin Nano Super 8GB + Ubuntu 22.04 + ROS2 Humble + Orbbec Femto Mega** 平台上，运行已经迁移好的 ROS2 版本 ORB-SLAM2 点云建图项目。

目标是：使用 Femto Mega 输出对齐后的 RGB-D 数据，送入 ORB-SLAM2 进行位姿估计和关键帧点云融合，最终保存房间三维点云模型。

🔗GitHub仓库

---

1. 项目复现目标 🎯

本项目复现的是一个基于 RGB-D 相机的室内三维重建流程：

```
代码块
1    Femto Mega RGB-D 相机
```

```
        ↓
pyorbbecsdk 深度对齐到彩色图
        ↓
ROS2 发布 aligned color/depth 话题
        ↓
ORB-SLAM2 RGB-D 跟踪、建图、关键帧管理
        ↓
点云融合并保存 result.pcd
        ↓
使用 pcl_viewer / CloudCompare 可视化模型
```



相比原 ROS1 项目，本次复现重点完成了：

- 将 ROS1 接入层迁移到 ROS2 Humble。
- 适配 Jetson Orin Nano + Ubuntu 22.04。

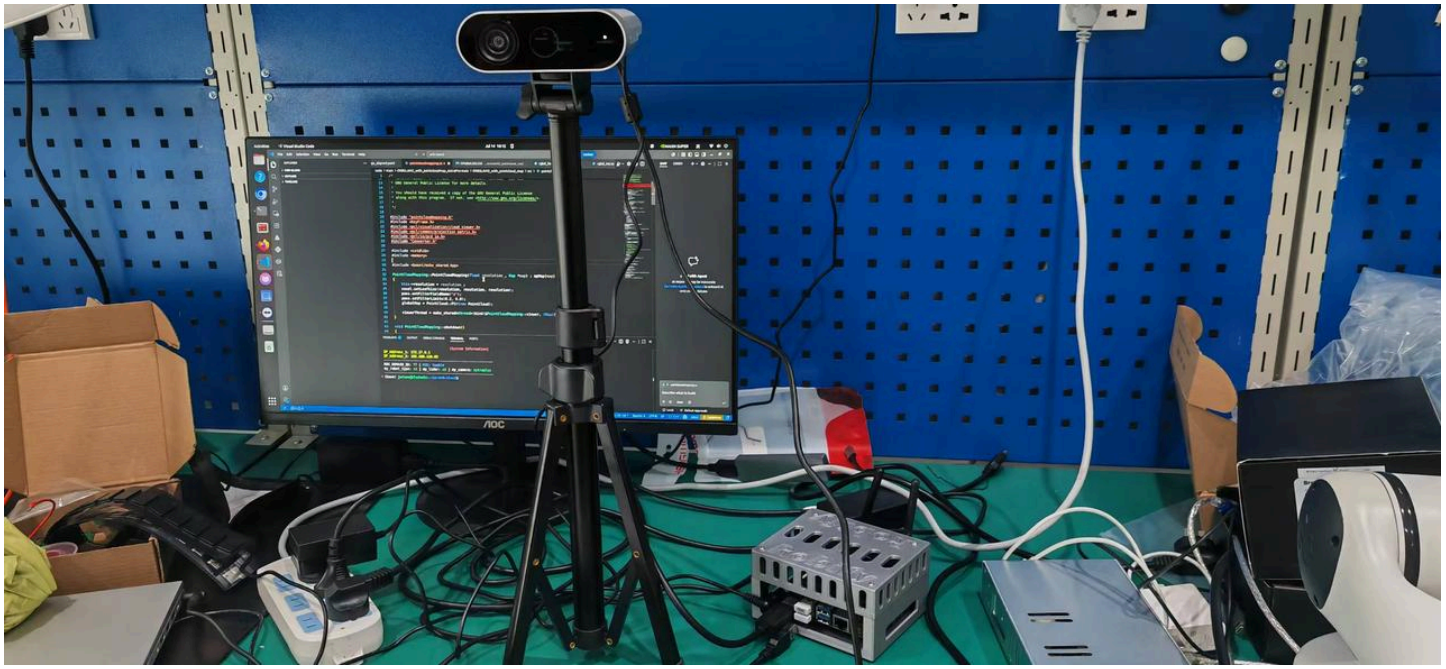
- 使用 `pyorbbecsdk` 自定义发布 **对齐后的 RGB-D 图像**。

- 解决官方 Orbbec ROS2 包没有对齐图像话题的问题。
- 移除 `cv_bridge`，降低 OpenCV 多版本混用导致崩溃的风险。
- 支持保存 `CameraTrajectory.txt`、`KeyFrameTrajectory.txt` 和 `result.pcd`。

---

2. 硬件与软件环境 🧩

2.1 硬件环境



项目	配置
开发板	Jetson Orin Nano Super 8GB
RGB-D 相机	Orbbec Femto Mega（可自行替换为Orbbec其他系列的深度相机，但同时要保证将相关相机内参数替换为通过官方SDK获取的参数值）
连接方式	USB / 网络连接，按实际设备配置
运行方式	手持相机扫描室内场景

2.2 软件环境

项目	推荐配置

操作系统	Ubuntu 22.04
ROS 版本	ROS2 Humble
Python	Python 3.10
相机 SDK	`pyorbbecsdk2`
点云库	PCL
SLAM 核心	ORB-SLAM2 with point cloud map
可视化工具	`pcl_viewer` / CloudCompare / MeshLab

---

3. 项目目录说明 🗂️

假设项目放置在：

```
/home/jetson/ws/orb-slam2/code
```

核心目录如下：

```
/home/jetson/ws/orb-slam2/code
├── main/
│   └── ORBSLAM2_with_pointcloudmap_AstraPro-main/
│       └── ORBSLAM2_with_pointcloud_map/
│           ├── bin/
│           ├── build.sh
│           ├── CameraTrajectory.txt
│           ├── cmake_modules/
│           ├── Dependencies.md
│           ├── Examples/
```

```
11  │            ├── include/
12  │            ├── KeyFrameTrajectory.txt
13  │            ├── lib/
14  │            ├── LICENSE.txt
15  │            ├── License-gpl.txt
16  │            ├── ORBvoc.txt.tar.gz
17  │            ├── pic/
18  │            ├── README.md
19  │            ├── script/
20  │            ├── src/
21  │            ├── Thirdparty/
22  │            └── Vocabulary/
23  │
24  └── ros2_orbslam2_pointcloud_map/
25      ├── docs/
26      ├── Log/
27      ├── README.md
28      ├── scripts/
29      │   ├── build_orbslam2_core.sh
30      │   ├── build_ros2_ws.sh
31      │   └── (其他脚本文件)
32      └── src/
33          └── orbslam2_pointcloud_ros2/
34              ├── CMakeLists.txt
35              ├── package.xml
36              ├── config/
37              │   ├── femto_mega.yaml
38              │   └── femto_mega_aligned.yaml
39              ├── launch/
40              │   ├── orbbec_aligned_rgbd_publisher.launch.py
41              │   └── rgbd_femto_mega.launch.py
42              ├── scripts/
43              │   └── orbbec_aligned_rgbd_publisher.py
44              └── src/
45                  └── rgbd_node.cpp
```

---

4. 环境准备 ⚙️

4.1 加载 ROS2 环境

代码块

```
source /opt/ros/humble/setup.bash
```

建议写入 `~/.bashrc`：

代码块

```
echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

4.2 安装常用依赖

代码块

```
sudo apt update

sudo apt install -y \
  build-essential cmake git wget unzip \
  ros-humble-rclcpp \
  ros-humble-rclpy \
  ros-humble-sensor-msgs \
  ros-humble-std-msgs \
  ros-humble-message-filters \
  python3-colcon-common-extensions \
  python3-pip \
  libpcl-dev \
  pcl-tools
```

4.3 安装 pyorbbecsdk

本项目的对齐 RGB-D 发布器依赖 `pyorbbecsdk`：

代码块

```
pip3 install --upgrade pyorbbecsdk2
```

验证：

代码块

```
python3 -c "from pyorbbecsdk import Pipeline; print('pyorbbecsdk OK')"
```

如果相机无法打开，可能需要安装 Orbbec udev 规则。进入 pyorbbecsdk 源码目录后执行：

代码块

```
cd ~/ws/pyorbbecsdk-2-main/scripts/env_setup
sudo chmod +x install_udev_rules.sh
sudo ./install_udev_rules.sh
```

然后重新插拔相机。

---

5. 设置 ORB-SLAM2 核心路径 🧠

进入工作区：

代码块

```
cd /home/jetson/ws/orb-slam2/code/ros2_orbslam2_pointcloud_map
```

设置 ORB-SLAM2 核心路径：

代码块

```
export ORB_SLAM2_ROOT=/home/jetson/ws/orb-slam2/code/main/ORBSLAM2_with_pointcloudmap_AstraPro-main/ORBSLAM2_with_pointcloud_map
```

建议写入 `~/.bashrc`：

代码块

```
echo "export ORB_SLAM2_ROOT=/home/jetson/ws/orb-slam2/code/main/ORBSLAM2_with_pointcloudmap_AstraPro-main/ORBSLAM2_with_pointcloud_map" >> ~/.bashrc
source ~/.bashrc
```

检查路径：

代码块

```
ls $ORB_SLAM2_ROOT/include/System.h
ls $ORB_SLAM2_ROOT/Vocabulary/ORBvoc.bin
```

---

6. 编译 ORB-SLAM2 点云核心 🔨

进入 ROS2 工作区：

```
代码块
1   cd /home/jetson/ws/orb-slam2/code/ros2_orbslam2_pointcloud_map
2   chmod +x scripts/*.sh
```

编译核心：

```
代码块
1   ./scripts/build_orbslam2_core.sh
```

编译成功后应存在：

```
代码块
1   ls $ORB_SLAM2_ROOT/lib/libORB_SLAM2.so
2   ls $ORB_SLAM2_ROOT/Thirdparty/DBoW2/lib/libDBoW2.so
3   ls $ORB_SLAM2_ROOT/Thirdparty/g2o/lib/libg2o.so
```

---

7. 编译 ROS2 迁移工程 🚀

```
代码块
1   cd /home/jetson/ws/orb-slam2/code/ros2_orbslam2_pointcloud_map
2   source /opt/ros/humble/setup.bash
3   ./scripts/build_ros2_ws.sh
4   source install/setup.bash
```

如果需要干净重编译：

```
代码块
1   rm -rf build install log
2   ./scripts/build_ros2_ws.sh
3   source install/setup.bash
```

---

8. 检查 OpenCV 链接版本 🧪

ORB-SLAM2 核心和 ROS2 节点应尽量使用同一套 OpenCV。

如果混入多个 OpenCV 版本，可能出现 `matrix.cpp: setSize` 、段错误、初始化后崩溃等问题。

检查：

```
代码块
1   ldd $ORB_SLAM2_ROOT/lib/libORB_SLAM2.so | grep opencv
2   ldd install/orbslam2_pointcloud_ros2/lib/orbslam2_pointcloud_ros2/rgbd_node |
    grep opencv
```

理想情况是只看到同一套 OpenCV，例如：

```
代码块
1   /usr/local/lib/libopencv_*.so.410
```

不建议出现类似混用：

```
代码块
1   /usr/local/lib/libopencv_*.so.410
2   /lib/aarch64-linux-gnu/libopencv_*.so.4.5d
3   /usr/local/lib/libopencv_core.so.3.2
```

---

9. 启动对齐 RGB-D 发布器 📷

官方 Orbbec ROS2 包可能只发布：

```
代码块
1   /camera/color/image_raw
2   /camera/depth/image_raw
```

但这两个话题可能分辨率不同，也不一定完成深度到彩色的对齐。

因此本项目新增了：

```
代码块
1   orbbec_aligned_rgbd_publisher.py
```

它会直接使用 `pyorbbecsdk` 发布对齐后的 RGB-D 话题。

9.1 重要提醒

运行本节点前，请关闭官方 `orbbec_camera` 节点，否则相机设备可能被占用。

9.2 启动发布器

终端 1：

代码块

```
cd /home/jetson/ws/orb-slam2/code/ros2_orbslam2_pointcloud_map
source /opt/ros/humble/setup.bash
source install/setup.bash

ros2 launch orbslam2_pointcloud_ros2 orbbec_aligned_rgbd_publisher.launch.py
```

正常日志应包含：

代码块

```
Using pyorbbecsdk software depth-to-color alignment.
Frame sync enabled.
RGB intrinsics for aligned depth: fx=... fy=... cx=... cy=...
Aligned stream ready: color/depth 1280x720, depth_scale=...
```

9.3 检查发布的话题

另开终端：

代码块

```
source /opt/ros/humble/setup.bash
```

```
source /home/jetson/ws/orb-slam2/code/ros2_orbslam2_pointcloud_map/install/setup.bash

ros2 topic list
```

应看到：

代码块

```
/orbbec/aligned/color/camera_info
/orbbec/aligned/color/image_raw
/orbbec/aligned/depth/camera_info
/orbbec/aligned/depth/image_raw
```

检查帧率：

代码块

```
ros2 topic hz /orbbec/aligned/color/image_raw
ros2 topic hz /orbbec/aligned/depth/image_raw
```

检查图像尺寸和编码：

代码块

```
ros2 topic echo /orbbec/aligned/color/image_raw --once | grep -E "height|width|encoding"
ros2 topic echo /orbbec/aligned/depth/image_raw --once | grep -E "height|width|encoding"
```

推荐结果：

```
代码块
1   color: height=720, width=1280, encoding=rgb8
2   depth: height=720, width=1280, encoding=16UC1
```

---

10. 更新相机内参 🗒️

启动发布器后，它会打印：

```
代码块
1   RGB intrinsics for aligned depth: fx=... fy=... cx=... cy=...
```

将这些值写入：

```
代码块
1   /home/jetson/ws/orb-
    slam2/code/ros2_orbslam2_pointcloud_map/src/orbslam2_pointcloud_ros2/config/fem
    to_mega_aligned.yaml
```

或安装后的配置：

```
代码块
1   /home/jetson/ws/orb-
    slam2/code/ros2_orbslam2_pointcloud_map/install/orbslam2_pointcloud_ros2/share/
    orbslam2_pointcloud_ros2/config/femto_mega_aligned.yaml
```

配置示例：

代码块

```
Camera.fx: 745.446
Camera.fy: 745.367
Camera.cx: 634.326
Camera.cy: 351.723

Camera.width: 1280
Camera.height: 720
Camera.fps: 30.0

Camera.RGB: 1
DepthMapFactor: 1000.0

rgb_topic: /orbbec/aligned/color/image_raw
depth_topic: /orbbec/aligned/depth/image_raw
```

说明：

- 对齐后的深度图已经投影到彩色相机坐标系。

- 因此配置文件应使用 **彩色相机内参**，不是原始深度相机内参。

- `DepthMapFactor: 1000.0` 对应 `16UC1` 毫米深度图。

---

11. 启动 ORB-SLAM2 RGB-D 建图 🗺️

确保终端 1 的对齐发布器正在运行。

然后打开终端 2：

代码块

```
cd /home/jetson/ws/orb-slam2/code/ros2_orbslam2_pointcloud_map
source /opt/ros/humble/setup.bash
source install/setup.bash
```

```
export ORB_SLAM2_ROOT=/home/jetson/ws/orb-slam2/code/main/ORBSLAM2_with_pointcloudmap_AstraPro-main/ORBSLAM2_with_pointcloud_map
```

启动 SLAM：

代码块

```
ros2 launch orbslam2_pointcloud_ros2 rgbd_femto_mega.launch.py \
  vocabulary_path:=$ORB_SLAM2_ROOT/Vocabulary/ORBvoc.bin \
  settings_path:=/home/jetson/ws/orb-slam2/code/ros2_orbslam2_pointcloud_map/install/orbslam2_pointcloud_ros2/share/orbslam2_pointcloud_ros2/config/femto_mega_aligned.yaml \
  output_dir:=/home/jetson/ws/orb-slam2/results \
  enable_viewer:=true \
  enable_pcl_viewer:=false
```

注意：`settings_path` 不要写成 `~/...`。

ROS2 launch 传参时，`~` 可能不会自动展开，建议使用完整路径 `/home/jetson/...`。

正常启动日志应包含：

代码块

```
RGB topic: /orbbec/aligned/color/image_raw
Depth topic: /orbbec/aligned/depth/image_raw
Vocabulary loaded
Camera Parameters:
Depth Threshold ...
Processed RGB-D frames: 1
receive a keyframe
show global map, size=...
```

---

12. 停止并保存结果 💾

扫描完成后，另开终端执行：

```
代码块
1  source /opt/ros/humble/setup.bash
2
3  ros2 topic pub --once /RGBD/cmd std_msgs/msg/String "{data: stop}"
```

输出文件位于：

```
代码块
1  /home/jetson/ws/orb-slam2/results/CameraTrajectory.txt
2  /home/jetson/ws/orb-slam2/results/KeyFrameTrajectory.txt
3  /home/jetson/ws/orb-slam2/results/result.pcd
```

检查：

```
代码块
1  ls -lh /home/jetson/ws/orb-slam2/results
2  wc -l /home/jetson/ws/orb-slam2/results/CameraTrajectory.txt
3  wc -l /home/jetson/ws/orb-slam2/results/KeyFrameTrajectory.txt
```

---

13. 可视化建模结果 👀

13.1 使用 pcl_viewer 查看点云

```
代码块
1   pcl_viewer /home/jetson/ws/orb-slam2/results/result.pcd
```

13.2 转换为 PLY

如果想用 CloudCompare / MeshLab 查看，可以转换为 `.ply`：

```
代码块
1   pcl_pcd2ply \
2     /home/jetson/ws/orb-slam2/results/result.pcd \
3     /home/jetson/ws/orb-slam2/results/result.ply
```

13.3 使用 CloudCompare

将文件拷贝到电脑：

```
代码块
1   scp jetson@<Jetson_IP>:/home/jetson/ws/orb-slam2/results/result.pcd .
```

然后用 CloudCompare 打开。

---

14. 完整房间建模的扫描建议 🏠

ORB-SLAM2 不是“拿到深度就无脑融合”的建模系统。

它需要稳定视觉跟踪、足够关键帧和连续视野重叠。

推荐扫描方式：

1. 启动后先让相机静止 2 到 3 秒。
2. 先对准有纹理的墙角、桌面、柜子、门框。
3. 不要一开始就扫纯白墙、地面或天花板。
4. 移动速度控制在每秒 10 到 20 cm。
5. 转动相机要慢，避免快速横扫。
6. 每一帧与上一帧保持约 60% 以上画面重叠。
7. 建议扫描顺序：墙面一圈 → 房间角落 → 家具/门窗 → 地面 → 房顶。
8. 如果墙面纹理太少，可以临时贴几张带图案的纸或棋盘格。

如果日志反复出现：

```
代码块
1   Track lost soon after initialisation, reseting...
2   System Reseting
```

说明系统一直在“初始化成功 → 立即丢失 → 清空地图重来”。

此时最终保存的数据通常只有几行，或者 `result.pcd` 很小。

---

15. 推荐参数调整 🛠️

如果跟踪容易丢失，可以尝试修改：

```
代码块
    /home/jetson/ws/orb-
    slam2/code/ros2_orbslam2_pointcloud_map/src/orbslam2_pointcloud_ros2/config/fem
    to_mega_aligned.yaml
```

推荐参数：

```
代码块
1   ORBextractor.nFeatures: 2000
2   ORBextractor.iniThFAST: 12
3   ORBextractor.minThFAST: 5
4   ThDepth: 80.0
```

如果房间更大，可以尝试：

```
代码块
1   ORBextractor.nFeatures: 2500
2   ThDepth: 100.0
```

如果只建出房顶或近距离区域，可以修改点云深度过滤。

源码文件：

```
代码块
1   /home/jetson/ws/orb-slam2/code/main/ORBSLAM2_with_pointcloudmap_AstraPro-
    main/ORBSLAM2_with_pointcloud_map/src/pointcloudmapping.cc
```

将：

```
代码块
pass.setFilterLimits(0.0, 3.0);
```

改为：

```
代码块
1   pass.setFilterLimits(0.2, 6.0);
```

修改后重新编译核心和 ROS2 工作区：

```
代码块
1   cd /home/jetson/ws/orb-slam2/code/ros2_orbslam2_pointcloud_map
2   ./scripts/build_orbslam2_core.sh
3   rm -rf build install log
4   ./scripts/build_ros2_ws.sh
5   source install/setup.bash
```

---

16. 常见问题排查表 🧯

现象	可能原因	解决办法
`package.xml` invalid XML	XML 中有非法字符或邮箱格式错误	用标准 XML 覆盖 `package.xml`，邮箱使用 `xxx@example.com`
`ApproximateTime()` 无参构造报错	Humble 下 `message_filters` 构造方式不兼容	使用 `sync_ = make_shared<Synchronizer>(policy); sync_->connectInput(...)`

`RcutilsLogger.info() takes 2 positional arguments`	Python rclpy 日志不支持 printf 风格参数	改成 f-string，例如 `logger.info(f"fx={fx}")`
`settings_path:=~/...` 打不开	launch 参数不展开 `~`	使用 `/home/jetson/...` 绝对路径
RGB 和 Depth 尺寸不同	官方 ROS2 驱动未发布对齐图像	使用 `orbbec_aligned_rgbd_publisher.py`
启动后没有帧被处理	对齐发布器未运行或话题不匹配	检查 `/orbbec/aligned/...` 话题和配置文件
运行时 OpenCV `setSize` 崩溃	OpenCV 多版本混用或 cv_bridge 牵入系统 OpenCV	检查 `ldd`，并使用已移除 `cv_bridge` 的节点
轨迹只有几行	跟踪频繁丢失并 reset	放慢移动速度，增加纹理，调高特征点数量
只重建出房顶	深度过滤范围太小或相机姿态偏上	放宽 `pass.setFilterLimits`，水平扫描墙面
`result.pcd` 不存在	全局点云为空	确认关键帧数量和 `show global map, size` 日志

---

17. 成功复现的判断标准 ✅

一次较好的房间扫描应满足：

代码块

```
1. /orbbec/aligned/color/image_raw 和 /orbbec/aligned/depth/image_raw 稳定发布
2. RGB 和 Depth 分辨率一致
3. SLAM 日志持续出现 Processed RGB-D frames
4. 不频繁出现 Track lost soon after initialisation
5. KeyFrameTrajectory.txt 至少有十几到几十行
6. result.pcd 存在且不是 0 字节
7. pcl_viewer 可以看到较完整的墙面、家具、房顶或地面点云
```

检查命令：

```
代码块
1  ros2 topic hz /orbbec/aligned/color/image_raw
2  ros2 topic hz /orbbec/aligned/depth/image_raw
3
4  wc -l /home/jetson/ws/orb-slam2/results/CameraTrajectory.txt
5  wc -l /home/jetson/ws/orb-slam2/results/KeyFrameTrajectory.txt
6  ls -lh /home/jetson/ws/orb-slam2/results/result.pcd
```

---

18. 最小运行命令汇总 🗒️

终端 1：启动对齐 RGB-D 发布器

```
代码块
1  cd /home/jetson/ws/orb-slam2/code/ros2_orbslam2_pointcloud_map
2  source /opt/ros/humble/setup.bash
3  source install/setup.bash
4
5  ros2 launch orbslam2_pointcloud_ros2 orbbec_aligned_rgbd_publisher.launch.py
```

终端 2：启动 ORB-SLAM2 RGB-D 建图

```
代码块
1  cd /home/jetson/ws/orb-slam2/code/ros2_orbslam2_pointcloud_map
2  source /opt/ros/humble/setup.bash
3  source install/setup.bash
4  export ORB_SLAM2_ROOT=/home/jetson/ws/orb-
   slam2/code/main/ORBSLAM2_with_pointcloudmap_AstraPro-
```

```
main/ORBSLAM2_with_pointcloud_map

ros2 launch orbslam2_pointcloud_ros2 rgbd_femto_mega.launch.py \
  vocabulary_path:=$ORB_SLAM2_ROOT/Vocabulary/ORBvoc.bin \
  settings_path:=/home/jetson/ws/orb-slam2/code/ros2_orbslam2_pointcloud_map/install/orbslam2_pointcloud_ros2/share/orbslam2_pointcloud_ros2/config/femto_mega_aligned.yaml \
  output_dir:=/home/jetson/ws/orb-slam2/results \
  enable_viewer:=false
```

终端 3：停止并保存

代码块

```
source /opt/ros/humble/setup.bash
ros2 topic pub --once /RGBD/cmd std_msgs/msg/String "{data: stop}"
```

查看点云：

代码块

```
pcl_viewer /home/jetson/ws/orb-slam2/results/result.pcd
```

---

19. 小结 🌟

本项目复现的关键不只是“跑起来”，而是要保证：

- RGB 与 Depth 已对齐；
- 相机内参使用对齐后的彩色相机内参；

- OpenCV 版本不要混用；
- ORB-SLAM2 跟踪过程不能频繁 reset；
- 扫描时需要慢速（每秒10～20cm）、重叠、纹理充足；
- 最终以 `result.pcd` 作为三维建模可视化结果。

只要对齐发布器稳定、SLAM 跟踪稳定、关键帧数量足够，Jetson Orin Nano + Femto Mega 就可以完成基础的室内 RGB-D 点云重建。